ECE276B Project 2 Report: 3D Pathfinding with A*

Stephen Huang
Department of Electrical and Computer Engineering
University of California, San Diego
La Jolla, U.S.A.
sth004@ucsd.edu

***Abstract*—We present a search-based motion planning algorithm for navigating in three-dimensional Euclidean environments with axis-aligned rectangular obstacles. The goal is to compute collision-free paths from a fixed start position to both static and dynamic goal locations. We first implement continuous collision checking between line segments and axis-aligned bounding boxes (AABBs) using the slab method. Then, we formulate the planning problem as a deterministic shortest path problem over a discretized 3D grid and solve it using a weighted A* search algorithm. The planner uses a 26-connected action space, Euclidean-distance heuristic and batched collision checking. Finally, we extend the same planner to dynamic target environments by replanning sequentially to each new goal position as it appears. Since our planner is efficient, we are able to produce collision-free paths**

I. Introduction

Motion planning is important in robotics because a robot must determine a feasible path from an initial state to a desired goal while avoiding obstacles in the environment. In many real-world environments, this is difficult because the robot moves in continuous space, obstacles may have complex geometry, and the target location may change over time.

In this project, we study motion planning in static, three-dimensional euclidean environments with a rectangular boundary and a set of axis-aligned rectangular obstacle blocks. Each planning problem specifies a start position and a static or sequence of dynamic goal positions. The objective is to compute a collision-free path from the start to the goal while remaining inside the environment boundary. For the dynamic target environments, the planner must repeatedly compute a path to the target before it moves to the next goal location.

To solve this problem, we use a grid-based weighted A* planner with continuous collision checking. Each candidate edge is checked as a continuous line segment against the rectangular obstacle blocks. This detects any possible collision along the path between adjacent grid states and allows the planner to reject unsafe actions before adding them to the graph.

Our approach formulates the continuous 3D space as a deterministic shortest path problem over a graph by discretizing the environment. Each vertex corresponds to a valid grid coordinate, and each edge corresponds to one of the 26 possible actions in 3D. The edge cost is the Euclidean length of the motion, and the heuristic is the Euclidean distance from the current coordinate to the goal.

For dynamic target environments, we implement a warm start A*. The first goal computation behaves identical to a normal A* search, but subsequent goals will use previously explored nodes with an updated heuristic. The planner receives goals sequentially, and has no knowledge of future goals.

II. Problem Formulation

Consider a robot navigating in a three-dimensional Euclidean environment. The environment is represented by a rectangular outer boundary and a finite set of axis-aligned bounding blocks (AABBs). The objective is to compute a collision-free path from a start position to a goal position while remaining inside the boundary.

Let the robot position at planning step t be

$$\mathbf{x}_t = \begin{bmatrix} x_t & y_t & z_t \end{bmatrix}^\top \in \mathbb{R}^3. \tag{1}$$

The start position is denoted by $\mathbf{x}_s \in \mathbb{R}^3$, and the goal position is denoted by $\mathbf{x}_g \in \mathbb{R}^3$. For dynamic target environments, the target is instead represented by a sequence of goal positions

$$\mathcal{G} = \{\mathbf{x}_{g,1}, \mathbf{x}_{g,2}, \ldots, \mathbf{x}_{g,N_g}\}, \tag{2}$$

where each $\mathbf{x}_{g,i} \in \mathbb{R}^3$ is the target location at a different time interval.

A. Environment Representation

The outer boundary is an axis-aligned bounding box defined by

$$B = [\mathbf{b}_{\min}, \mathbf{b}_{\max}], \tag{3}$$

where

$$\mathbf{b}_{\min} = \begin{bmatrix} x_{\min} & y_{\min} & z_{\min} \end{bmatrix}^\top, \tag{4}$$

$$\mathbf{b}_{\max} = \begin{bmatrix} x_{\max} & y_{\max} & z_{\max} \end{bmatrix}^\top. \tag{5}$$

The obstacle set consists of M axis-aligned rectangular blocks:

$$O = \{O_1, O_2, \ldots, O_M\}. \tag{6}$$

Each obstacle O_j is also represented as an axis-aligned bounding box,

$$O_j = [\mathbf{o}_{j,\min}, \mathbf{o}_{j,\max}], \tag{7}$$

with lower and upper corners $\mathbf{o}_{j,\min}, \mathbf{o}_{j,\max} \in \mathbb{R}^3$.

B. Free Space and Valid Paths

The free space is the set of all points inside the boundary and outside all obstacles:

$$\mathcal{X}_{\text{free}} = \left\{ \mathbf{x} \in \mathbb{R}^3 \;\middle|\; \mathbf{x} \in B,\ \mathbf{x} \notin \bigcup_{j=1}^{M} O_j \right\}. \tag{8}$$

A continuous path is a function

$$t : [0,1] \to \mathcal{X}_{\text{free}}, \tag{9}$$

where

$$t(0) = \mathbf{x}_s, \qquad t(1) = \mathbf{x}_g. \tag{10}$$

We consider a path feasible if every point in the path lies in free space

$$t(x) \in \mathcal{X}_{\text{free}}, \qquad \forall x \in [0,1]. \tag{11}$$

C. Deterministic Shortest Path

We formulate the continuous 3D environment as a discrete grid with resolution $R > 0$ Each voxel corresponds to a graph vertex. We define the graph

$$G = (V, E), \tag{12}$$

where V is the set of valid grid coordinates and E is the set of weighted actions.

We define the control space as

$$\mathcal{U} = \{R(\Delta i, \Delta j, \Delta k) \mid \Delta i, \Delta j, \Delta k \in \{-1, 0, 1\}, \tag{13}$$

$$(\Delta i, \Delta j, \Delta k) \neq (0,0,0)\}. \tag{14}$$

Thus, each node has at most 26 possible actions.

The deterministic transition model is

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t) = \mathbf{x}_t + \mathbf{u}_t, \tag{15}$$

where $\mathbf{u}_t \in \mathcal{U}$. A transition is valid only if the segment from $\mathbf{x}_t$ to $\mathbf{x}_{t+1}$ remains inside the boundary and does not intersect any obstacle.

The stage cost is the Euclidean length of the action:

$$\ell(\mathbf{x}_t, \mathbf{u}_t) = \|\mathbf{u}_t\|_2. \tag{16}$$

Therefore, axis-aligned moves have cost R, two-axis diagonal moves have cost $\sqrt{2}R$, and three-axis diagonal moves have cost $\sqrt{3}R$.

The objective is to find a feasible control sequence

$$\mathbf{u}_{0:K-1} = (\mathbf{u}_0, \mathbf{u}_1, \ldots, \mathbf{u}_{K-1}) \tag{17}$$

that minimizes the total path length:

$$J(\mathbf{u}_{0:K-1}) = \sum_{t=0}^{K-1} \ell(\mathbf{x}_t, \mathbf{u}_t), \tag{18}$$

subject to

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t), \qquad \mathbf{x}_t \in \mathcal{X}_{\text{free}}, \qquad \mathbf{x}_K = \mathbf{x}_g. \tag{19}$$

For dynamic target environments, the planner solves a sequence of deterministic shortest path problems:

$$P_i = \text{Plan}(\mathbf{x}_s, \mathbf{x}_{g,i}), \qquad i = 1, \ldots, N_g. \tag{20}$$

III. Technical Approach

We solve the 3D motion planning problem using a search-based planner with continuous collision checking. We use slab collision checking with an A* search algorithm and warm restart for dynamic goals.

A. Agent Size

To prevent the agent from going between two connected walls and gliding along the surface of walls, we model the agent with a safety radius

$$r = 0.1. \tag{21}$$

To do this, we approximate the agent as a sphere of radius r centered at the robot position $\mathbf{x}_t$. Then, a path is considered safe if the center of the agent remains at least distance r away from all obstacles.

Equivalently, we inflate the obstacle blocks by the radius $r = 0.1$ by setting

$$\mathbf{o}'_{min} = \mathbf{o}_{min} - r, \qquad \mathbf{o}'_{max} = \mathbf{o}_{max} + r. \tag{22}$$

B. Collision Checking

We employ two methods to ensure the agent does not collide with AABBs or the boundary. First, we use an optimized slab method for ray-box collision detection. Then, we also check if the line segment is within the boundary.

1) Slab Method: Given a line segment from $\mathbf{p}_0$ to $\mathbf{p}_1$, the parametric line is

$$\mathbf{p}(t) = \mathbf{p}_0 + t\mathbf{d}, \qquad t \in [0,1], \tag{23}$$

where

$$\mathbf{d} = \mathbf{p}_1 - \mathbf{p}_0. \tag{24}$$

For an inflated AABB with lower corner $\mathbf{o}'_{\min}$ and upper corner $\mathbf{o}'_{\max}$, we compute the t values for which the segment lies inside each infinite slab. For each axis $a \in \{x, y, z\}$,

$$t_{a,\min} = \frac{o_{\min,a} - p_{0,a}}{d_a}, \qquad t_{a,\max} = \frac{o_{\max,a} - p_{0,a}}{d_a}. \tag{25}$$

Since the direction may be positive or negative, we define

$$t_{a,\text{near}} = \min(t_{a,\min}, t_{a,\max}), \quad t_{a,\text{far}} = \max(t_{a,\min}, t_{a,\max}). \tag{26}$$

The segment enters the box at

$$t_{\text{enter}} = \max_{a \in \{x,y,z\}} t_{a,\text{near}}, \tag{27}$$

and exits the box at

$$t_{\text{exit}} = \min_{a \in \{x,y,z\}} t_{a,\text{far}}. \tag{28}$$

The segment intersects the box if and only if

$$t_{\text{enter}} \leq t_{\text{exit}}, \qquad t_{\text{exit}} \geq 0, \qquad t_{\text{enter}} \leq 1. \tag{29}$$

In the implementation, collision checking accounts for the majority of computation time, especially during successor generation. Therefore, we vectorize the collision checking to reduce function call overhead in our implementation. This reduces 26 function calls for successor generation to one call.

2) Boundary Checking: The boundary is also an axis-aligned box. Since the boundary is rectangular, and therefore convex, a line segment remains inside the boundary if both endpoints are inside the boundary. Thus, for a segment $[\mathbf{p}_0, \mathbf{p}_1]$, the planner checks

$$\mathbf{b}_{\min} \leq \mathbf{p}_0 \leq \mathbf{b}_{\max}, \qquad \mathbf{b}_{\min} \leq \mathbf{p}_1 \leq \mathbf{b}_{\max}. \tag{30}$$

If either endpoint is outside the boundary, the successor is rejected.

C. Environment Construction

We initialize the environment with several known parameters:

- the boundary $[\mathbf{b}_{min}, \mathbf{b}_{max}]$
- the blocks $\{[\mathbf{o}'_{1,min}, \mathbf{o}'_{1,max}], \ldots, [\mathbf{o}'_{M,min}, \mathbf{o}'_{M,max}]\}$,
- the start $\mathbf{x}_s$,
- the goal $\mathbf{x}_g$,
- and the resolution R.

1) Action Initialization: From each grid state, the planner considers 26 possible actions:

$$\mathbf{u} \in R\{(-1,-1,-1), \ldots, (1,1,1)\} \setminus \{\mathbf{0}\}. \tag{31}$$

These actions have associated cost

$$c(\mathbf{u}) = \|\mathbf{u}\|_2. \tag{32}$$

These are pre-computed once on environment initialization.

A successor coordinate is generated as

$$\mathbf{x}' = \mathbf{x} + \mathbf{u}. \tag{33}$$

The successor is valid if:

1) the segment $[\mathbf{x}, \mathbf{x}']$ remains inside the boundary, and
2) the segment $[\mathbf{x}, \mathbf{x}']$ does not collide with any inflated obstacle.

Only valid successors are returned to the planner.

2) Heuristic Function: The heuristic function is the Euclidean distance from the current node to the goal:

$$h(\mathbf{x}_t, \mathbf{x}_g) = \|\mathbf{x}_t - \mathbf{x}_g\|_2. \tag{34}$$

This heuristic is admissible and consistent in this 3D space when $\epsilon = 1$ and also provides information on the distance to the goal assuming there are no obstacles. Therefore, the planner will be more inclined to choose nodes closer to the goal. This heuristic is computed for each node when it is created or updated in the graph.

3) Goal Checking: Since it is infeasible for the current grid coordinate to exactly equal the goal coordinate, we instead checks whether the current coordinate can be connected directly to the goal through a collision-free line segment. Given the current coordinate $\mathbf{x}$ and goal $\mathbf{x}_g$, we define the goal-check condition as

$$\text{IsGoal}(\mathbf{x}) = \text{InBoundary}(\mathbf{x}, \mathbf{x}_g) \text{ and not } \text{Collision}(\mathbf{x}, \mathbf{x}_g), \tag{35}$$

This allows the final path to reach the continuous goal coordinate instead of requiring the goal to lie exactly on the grid. It also improves efficiency and maintains optimality as the search can terminate as it can terminate earlier and follows the shortest path to the goal.

D. Node Representation and Grid Hashing

Each search node stores its key, grid coordinate, cost-to-come, heuristic value, parent pointer, and search status. Let node n correspond to coordinate $\mathbf{x}_n$. The cost-to-come is

$$g(n), \tag{36}$$

and the heuristic cost-to-go estimate is

$$h(n) = \|\mathbf{x}_n - \mathbf{x}_g\|_2. \tag{37}$$

Because floating-point coordinates can introduce small numerical differences, each coordinate is mapped to an integer grid key:

$$\text{key}(\mathbf{x}) = \text{round}\left(\frac{\mathbf{x}}{R}\right). \tag{38}$$

This ensures that repeated arrivals at the same grid cell are stored as the same graph vertex.

E. Weighted A* Search

We use a weighted A* search algorithm to compute a path from the start to the goal. The priority of a node is defined by

$$f(n) = g(n) + \epsilon h(n), \tag{39}$$

where ϵ is the heuristic weight. When $\epsilon = 0$, we have Dijkstra's algorithm. When $\epsilon = 1$, we have standard A*. When $\epsilon > 1$, we have greedier search, which may lead to less explored nodes and a suboptimal path.

The OPEN list is implemented as a priority queue. Each time a node is expanded, all valid successors are generated. If a successor has not been seen before, it is added to the graph. If a lower-cost path to a successor is found, we relabel:

$$g(n') \leftarrow g(n) + c(n, n'), \tag{40}$$

whenever

$$g(n) + c(n, n') < g(n'). \tag{41}$$

The successor is then inserted into or updated in the OPEN list with priority

$$f(n') = g(n') + \epsilon h(n'). \tag{42}$$

Algorithm 1 Weighted A* With Collision Checking

function PLAN($\mathbf{x}_s, \mathbf{x}_g, B, O, R, \epsilon$)
  Initialize graph dictionary G
  Initialize priority queue OPEN
  Create start node n_s
  $g(n_s) \leftarrow 0$
  $h(n_s) \leftarrow \|\mathbf{x}_s - \mathbf{x}_g\|_2$
  Insert n_s into OPEN
  while OPEN is not empty **do**
    $n \leftarrow$ node in OPEN with minimum $g(n) + \epsilon h(n)$
    if LineOfSight($\mathbf{x}_n, \mathbf{x}_g$) **then**
      Connect $\mathbf{x}_n$ directly to $\mathbf{x}_g$
      return recovered path
    end if
    Mark n as closed
    for each action $\mathbf{u} \in \mathcal{U}$ **do**
      $\mathbf{x}' \leftarrow \mathbf{x}_n + \mathbf{u}$
      if $[\mathbf{x}_n, \mathbf{x}']$ is outside boundary **then**
        continue
      end if
      if $[\mathbf{x}_n, \mathbf{x}']$ intersects an obstacle **then**
        continue
      end if
      $n' \leftarrow$ node associated with $\mathbf{x}'$
      $g_{\text{new}} \leftarrow g(n) + \|\mathbf{u}\|_2$
      if $g_{\text{new}} < g(n')$ **then**
        $g(n') \leftarrow g_{\text{new}}$
        parent(n') $\leftarrow n$
        Insert or update n' in OPEN
      end if
    end for
  end while
  return failure
end function

F. Direct Line-of-Sight Goal Connection

A major modification from vanilla A* is that the planner checks whether the current node can connect directly to the exact goal position. Specifically, for the current node coordinate $\mathbf{x}_n$, the planner checks whether the segment

$$[\mathbf{x}_n, \mathbf{x}_g] \tag{43}$$

is inside the boundary and collision-free. If this condition is true, the planner appends the exact goal position to the path and terminates.

This has two advantages. First, it allows the final path to end at the continuous goal coordinate rather than only at the nearest grid point. Second, it can reduce the number of node expansions because the planner does not need to search until a grid coordinate exactly equals the goal. This path is also optimal because it is a straight line and thus the shortest path.

G. Part 2 Analysis

For part 2, we implemented the above algorithms and focused on reducing the number of function calls, which takes up the majority of the computation time. Since our weighted A* used $\epsilon = 1.5$ we cannot guarantee an optimal path. We choose $R = 0.5$ as it is a good balance between speed and accuracy. The number of nodes scales inverse cubic to the resolution. This means that $R = 0.25$ takes $2^3 = 16$ times longer to compute than $R = 0.5$. Furthermore, since our computation speed was quick, we chose $\epsilon = 1$ to preserve optimality.

The main methods we used to scale to larger maps is to reduce repeated computation and function calls. That is, we pre-compute the action and cost vector and also vectorize collision checking. This reduced function calls from 22 million to 4 million. Furthermore, for larger maps, we can also increase the resolution size to reduce computation time, depending on how complex the map is.

Our algorithm does not guarantee completeness at all resolution scales. Especially on $R = 0.4$ and $R = 0.2$, where $R = 0.4$ fails on the Monza map, and $R = 0.2$ fails on the flappy bird map. We believe this is because those resolutions cause nodes to appear near or inside the walls during critical turns, creating "ghost" blockages that prevent a path from appearing.

For heuristic function, we decided to use Euclidean distance instead of octile distance. Although octile distance does accurately represent the distance with our 26-action control space, it led to increased computation time and path length.

One big difference from traditional A* and important implementation for time efficiency is the goal line-of-sight checker. Since many maps are open, we are able to significantly reduce computation time and expand far less nodes because of this implementation. However, Maze maps make this implementation into an overhead that might increase computation time. The goal checker's efficiency also improves as R decreases, which makes it scale effectively to large maps.

Overall, the part 2 planner performed well because the largest computational bottleneck was addressed through vectorization and precomputation. The collision checker remained continuous, so each edge was checked as a true line segment rather than only checking endpoint occupancy. This allowed the planner to produce safer paths while still using an efficient graph structure.

*H. Warm-Start A**

For dynamic target environments, the target moves through a sequence of goal positions. Rather than naively recomputing the path from scratch for each goal, we assume that the goals share large portions of useful nodes. Thus, to improve replanning speed, we extend the planner with a warm-start mechanism.

The warm-start planner stores the graph generated during the previous search. This graph contains previously created nodes, valid successor relationships implicitly represented by their grid coordinates, cost-to-come values, and parent pointers. When a new goal is received, it reuses the existing graph and updates the heuristic values according to the new goal:

$$h_i(n) = \|\mathbf{x}_n - \mathbf{x}_{g,i}\|_2, \tag{44}$$

where $\mathbf{x}_{g,i}$ is the current target position.

The priority of each reusable node is then recomputed as

$$f_i(n) = g(n) + \epsilon h_i(n). \tag{45}$$

This allows the search to begin with a graph that already contains many of the states explored in previous planning queries. Since the environment geometry does not change, we do not have to recompute collision for new grid nodes and transitions.

To avoid incorrectly treating the previous search as complete for the new goal, the planner resets the OPEN/CLOSED status of the nodes before each new query. The start node is inserted into the OPEN list, and previously discovered nodes can be reused if they are reached again with an improved cost. We also reset the entire graph once we detect a new environment.

Algorithm 2 Warm-Start A*

function WARMSTARTASTAR($G, \mathbf{x}_s, \mathbf{x}_{g,i}, B, O, R, \epsilon$)
  Initialize empty priority queue OPEN
  for each node $n \in G$ **do**
    $h(n) \leftarrow \|\mathbf{x}_n - \mathbf{x}_{g,i}\|_2$
    Reset OPEN/CLOSED status of n
  end for
  Get or create start node n_s
  $g(n_s) \leftarrow 0$
  Insert n_s into OPEN with priority $g(n_s) + \epsilon h(n_s)$
  while OPEN is not empty **do**
    $n \leftarrow$ node in OPEN with minimum $g(n) + \epsilon h(n)$
    if LineOfSight($\mathbf{x}_n, \mathbf{x}_{g,i}$) **then**
      Connect $\mathbf{x}_n$ directly to $\mathbf{x}_{g,i}$
      return recovered path and updated graph G
    end if
    Mark n as closed
    Generate valid successors using boundary and slab collision checks
    for each valid successor n' **do**
      $g_{\text{new}} \leftarrow g(n) + c(n, n')$
      if n' is new or $g_{\text{new}} < g(n')$ **then**
        $g(n') \leftarrow g_{\text{new}}$
        parent(n') $\leftarrow n$
        Insert or update n' in OPEN
      end if
    end for
  end while
  return failure
end function

This approach is similar to incremental methods because it reuses information across planning queries. However, it is simpler than LPA* or D* Lite.

I. Dynamic Target Planning

For the dynamic target environments, the planner receives a sequence of target positions

$$\{\mathbf{x}_{g,1}, \mathbf{x}_{g,2}, \ldots, \mathbf{x}_{g,8}\}. \tag{46}$$

For each target, the warm-start planner computes a path from the same start position to the current goal:

$$P_i = \text{WarmStartPlan}(\mathbf{x}_s, \mathbf{x}_{g,i}). \tag{47}$$

After each query, the updated graph is retained and passed into the next planning query:

$$G_i \rightarrow G_{i+1}. \tag{48}$$

Thus, the planner avoids rebuilding all search nodes from scratch for every target.

J. Part 3 Analysis

For part 3, we use the same weighted A* planner as part 2 with $\epsilon = 1.5$ and $R = 0.5$. The main difference is that the planner now supports warm-start replanning for multiple goals in the same environment. This improves computation time because the planner can reuse nodes that were already created in previous searches. This creates a tradeoff between memory and time efficiency. Since the graph persists across planning queries, it grows as the number of queries increases for the same environment. However, this also decreases the replanning speed as more valid nodes are already discovered. In the dynamic maps, this tradeoff is reasonable because each environment contains eight goal queries, which makes rapid replanning more important than memory.

The warm-start planner is still suboptimal because it uses $\epsilon = 1.5$, which makes the search greedier than standard A*. This reduces the number of expanded nodes but may increase path length. It also remains resolution-complete only with respect to the chosen grid resolution. Similarly, completeness is not guaranteed with different resolutions, but for $R = 0.5$, it will discover a path. Overall, warm-start improves fast replanning by reusing previous information but results in higher memory usage.

IV. RESULTS

The static environments are `E1_flappy_bird`, `E2_maze`, `E3_monza`, `E4_single_cube`, and `E5_tower`. The dynamic target environments are `E6_window` and `E7_room`.

A. Path Graph Results

The following images are the static environment path.

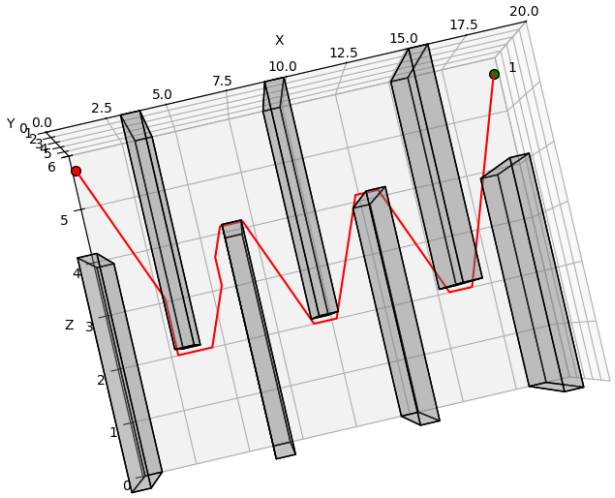


Fig. 1. Planned path for `E1_flappy_bird`.

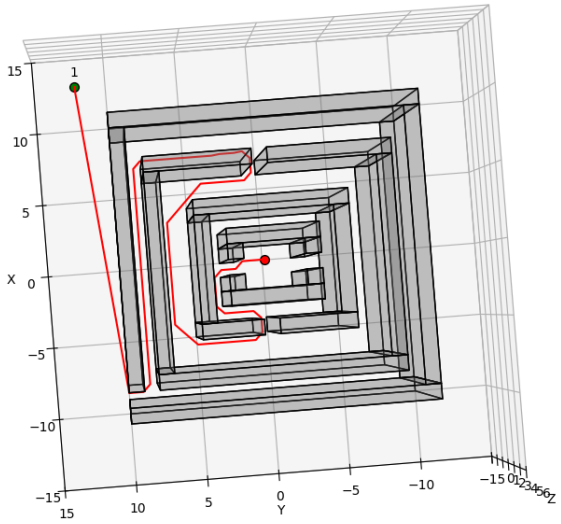


Fig. 2. Planned path for `E2_maze`.

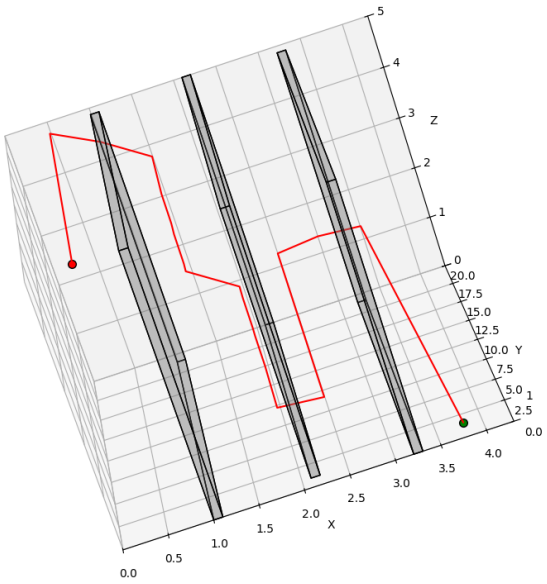


Fig. 3. Planned path for `E3_monza`.

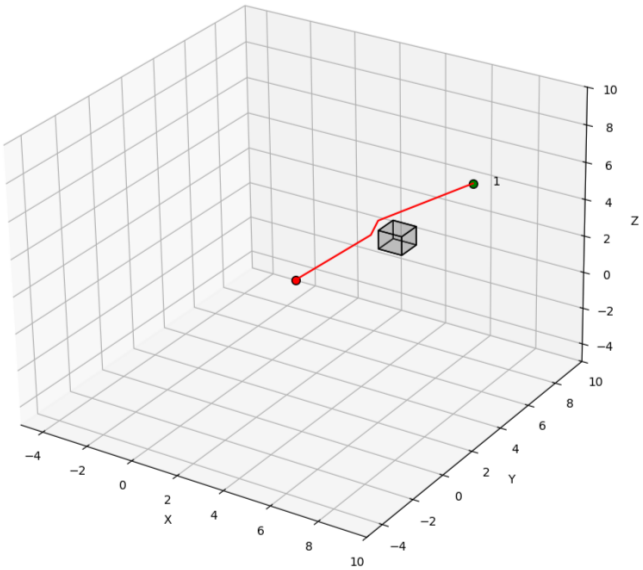


Fig. 4. Planned path for `E4_single_cube`.

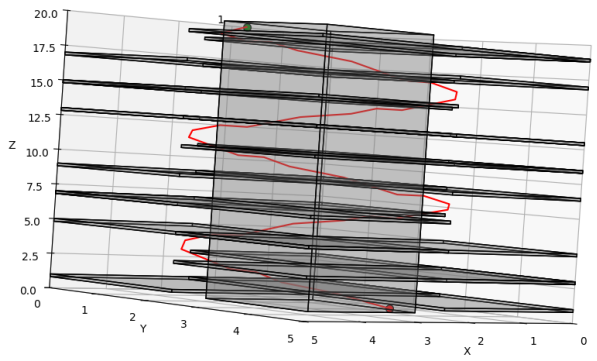


Fig. 5. Planned path for `E5_tower`.

The following images are the dynamic target paths.

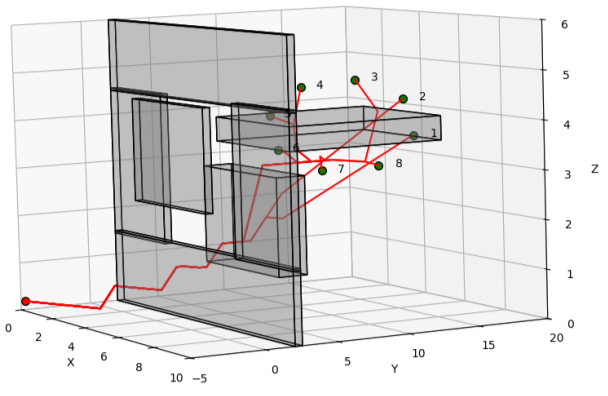


Fig. 6. Planned paths for the dynamic target sequence in `E6_window`.

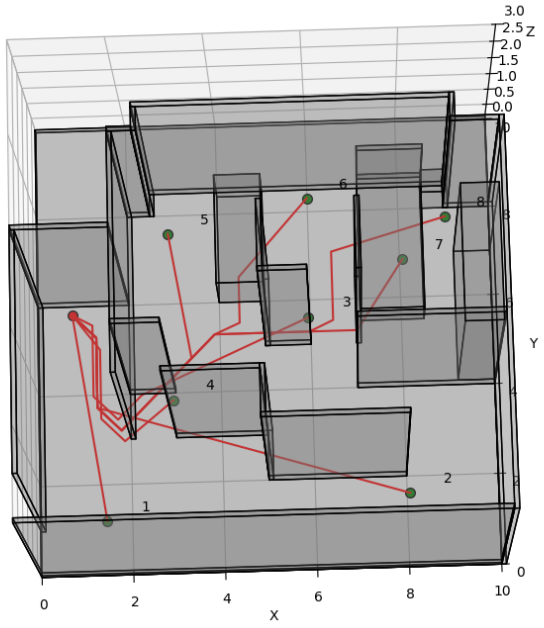


Fig. 7. Planned paths for the dynamic target sequence in `E7_room`.

B. Parameter Results

In addition to the graphs, we also evaluate different parameter settings on the planner using computation time, path distance, and number of expanded nodes. Each entry in the following tables is reported as

$$(\text{time}, \text{distance}, \text{nodes})$$

where time is measured in seconds.

TABLE I
STATIC MAP WARM-START A*, R=0.5.

Map	Time (s)	Path Dist.	Nodes Expanded
`E4_single_cube`	0.0020	8	10
`E2_maze`	1.0451	78	6955
`E1_flappy_bird`	0.4494	25	3384
`E3_monza`	0.2877	77	2450
`E5_tower`	0.3465	33	2446

Table I shows that the planner successfully computes collision-free paths for all five static maps at $R = 0.5$. The number of expanded nodes increases with the complexity and size of the environment.

TABLE II
HEURISTIC COMPARISON FOR STATIC MAP $R = 0.5$ AND $\epsilon = 1$.

Heuristic	Single Cube	Maze	Flappy Bird	Monza	Tower
Euclid	(0.0020, 8, 10)	(1.0451, 78, 6955)	(0.4494, 25, 3384)	(0.2877, 77, 2450)	(0.3465, 33, 2446)
Octile	(0.0112, 8, 20)	(2.7590, 78, 6955)	(1.1922, 25, 2936)	(0.8016, 78, 2447)	(0.9295, 33, 2329)

Table II compares Euclidean and octile heuristics. Although octile distance better matches a 26-connected grid, Euclidean distance produced lower computation time and path length.

TABLE III
RESOLUTION COMPARISON WARM-START EUCLIDEAN A*, $R = 0.5$

Resolution	Single Cube	Maze	Flappy Bird	Monza	Tower
0.1	(0.0410, 8, 82)	(366.7206, 73, 1116173)	(96.2201, 25, 324662)	(100.2025, 75, 283395)	(120.6918, 28, 254149)
0.3	(0.0109, 8, 22)	(18.2341, 74, 41691)	(4.9576, 26, 12066)	(3.4088, 76, 10174)	(4.3283, 29, 10045)
0.5	(0.0050, 8, 10)	(2.8599, 78, 6955)	(1.3026, 25, 3384)	(0.7994, 77, 2450)	(0.9391, 33, 2446)
0.7	(0.0042, 8, 8)	(1.0884, 76, 2804)	(0.3734, 27, 905)	(0.2599, 77, 808)	(0.3143, 32, 797)
0.9	(0.0012, 8, 6)	(0.1957, 77, 1561)	(0.0518, 29, 437)	(0.0370, 80, 417)	(0.0464, 32, 362)

Table III shows tradeoff between resolution, path quality, and computation time. Smaller resolutions produce shorter paths, but the number of expanded nodes grows polynomially. We therefore choose $R = 0.5$ as a balance between path quality and runtime.

TABLE IV
WEIGHT COMPARISON FOR EUCLIDEAN STATIC A*, $R = 0.5$.

ϵ	Single Cube	Maze	Flappy Bird	Monza	Tower
1.5	(0.0033, 8, 6)	(2.7586, 79, 6935)	(0.4322, 26, 1027)	(0.8062, 78, 2446)	(0.6634, 33, 1626)
2.0	(0.0039, 8, 6)	(2.8711, 80, 6894)	(0.3001, 26, 723)	(0.7927, 78, 2440)	(0.1841, 34, 443)
2.5	(0.0010, 8, 6)	(2.8752, 80, 6869)	(0.2841, 27, 699)	(0.7996, 78, 2440)	(0.1379, 36, 335)
3.0	(0.0011, 8, 6)	(2.8608, 80, 6842)	(0.2619, 28, 628)	(0.8128, 78, 2437)	(0.0908, 37, 310)
5.0	(0.0032, 8, 6)	(2.7715, 81, 6780)	(0.2259, 30, 564)	(0.7965, 78, 2437)	(0.1192, 39, 295)

Table IV shows that larger ϵ generally reduce the number of expanded nodes and computation time. However, paths become longer. For example, in the tower map, increasing ϵ from 1.5 to 5.0 reduces the number of expanded nodes from 1626 to 295, but increases path distance from 33 to 39.

TABLE V
WEIGHT COMPARISON FOR EUCLIDEAN DYNAMIC A*, $R = 0.5, \epsilon = 1.5$.

ϵ	Window	Room
1.5	(0.0909, 176.64, 148)	(0.1683, 67.43, 431)
2.0	(0.0789, 182.44, 117)	(0.1195, 67.79, 288)
2.5	(0.0995, 183.40, 153)	(0.1072, 71.17, 256)
3.0	(0.0494, 177.75, 178)	(0.1031, 71.72, 226)
5.0	(0.0294, 178.80, 30)	(0.0718, 74.63, 169)

Table V shows that increasing ϵ can reduce total time and expanded nodes, but generally increases path distance. We use $\epsilon = 1.5$ because it gives a good balance between speed and path quality.

TABLE VI
WARM-START AND COLD-START COMPARISON DYNAMIC TARGET, $R = 0.5$

Planner	Heuristic / ϵ	Window	Room
Warm Start	Euclid, $\epsilon = 1$	(1.0445, 157.39, 4455)	(0.4029, 66.79, 947)
Cold Start	Euclid, $\epsilon = 1$	(1.8213, 157.39, 4455)	(0.4014, 66.79, 947)
Warm Start	Octile, $\epsilon = 1$	(1.4186, 159.66, 3390)	(0.3293, 66.97, 811)
Cold Start	Octile, $\epsilon = 1$	(1.3018, 159.66, 3214)	(0.3230, 66.97, 811)
Warm Start	Euclid, $\epsilon = 1.5$	(0.0909, 176.64, 148)	(0.1683, 67.43, 431)
Cold Start	Euclid, $\epsilon = 1.5$	(0.2185, 162.05, 526)	(0.2273, 67.43, 562)
Warm Start	Octile, $\epsilon = 1.5$	(0.0453, 167.62, 53)	(0.1702, 69.03, 414)
Cold Start	Octile, $\epsilon = 1.5$	(0.1152, 165.27, 239)	(0.2084, 68.80, 516)

Table VI compares warm-start and cold-start replanning for the dynamic target environments, along with different heuristics. It can be seen that across all Euclidean heuristics, warm-start significantly decreases the computation time.

TABLE VII
DYNAMIC TARGET TIMING WITH WARM-START EUCLIDEAN A*, $R = 0.5, \epsilon = 1.5$.

Map	t_1 (s)	t_2 (s)	$t_1 + 7t_2$ (s)
Window	0.0109	0.0517	0.3728
Room	0.0002	0.0408	0.2858

Table VII shows the minimum achievable timings for replanning, which is far faster than the current specifications.

C. Part 2 Static Planning Results

For the main static planning experiments, we use a Euclidean heuristic with resolution $R = 0.5$. At this resolution, the warm-start planner solves all five static maps regardless of complexity. For example, the single cube map requires only 10 expanded nodes, while the maze map requires 6955 expanded nodes because the planner must search through a much larger region. At $R = 0.5$, the warm-start Euclidean planner obtains path lengths of 8, 78, 25, 77, and 33 for the single cube, maze, flappy bird, Monza, and tower maps respectively. The corresponding expanded node counts are 10, 6955, 3384, 2450, and 2446. This shows that path quality and runtime depend strongly on the complexity of the map rather than physical map size.

The number of expanded nodes also explains the computation time. Maps with more complexity require the search to explore more nodes before a line-of-sight connection to the goal is found. The maze and flappy bird maps have the largest search costs because their obstacle structures force the planner to explore many candidate paths before finding a feasible route. In contrast, the single cube environment is solved almost immediately because there is a simple path around the central obstacle.

D. Parameter Analysis

We evaluate the effect of three parameters: grid resolution, heuristic choice, and heuristic weight. These results are summarized in the uploaded result tables.

1) Effect of Resolution: The grid resolution has the largest effect on computation time and node expansion. Smaller resolutions create denser graphs, which can produce slightly shorter paths but greatly increase the number of nodes. For example, on the maze map, warm-start Euclidean A* expands 1,116,173 nodes at $R = 0.1$, but only 6955 nodes at $R = 0.5$ and 1561 nodes at $R = 0.9$. This confirms that the number of grid states grows rapidly as the resolution becomes smaller.

However, larger resolutions can reduce path quality because the planner has fewer possible waypoints. At $R = 0.1$, the maze path length is 73, while at $R = 0.5$ it is 78, and at $R = 0.9$ it is 77. Thus, smaller resolutions can provide better geometric flexibility, but the computation cost is much higher. We chose $R = 0.5$ because it gives a good balance between computation time and path quality.

The resolution results also show that the planner is not complete across all resolution values. At $R = 0.2$ and $R = 0.4$, some maps return no path. This occurs because grid

alignment can remove usable nodes from narrow passages or critical turns, even when a continuous path exists. Therefore, the planner is resolution-complete with respect to the selected graph, but not complete in continuous space.

2) Effect of Heuristic Choice: We compare Euclidean and octile heuristics at $R = 0.5$. For the unweighted planner, both heuristics produce similar path lengths, but Euclidean distance generally gives lower computation time in the static maps. For example, with warm start, Euclidean A* solves the flappy bird map in 0.4494s while octile requires 1.1922s. Similarly, Euclidean solves Monza in 0.2877s while octile requires 0.8016s.

Although octile distance matches the control space more directly, it did not consistently improve runtime in our implementation. Since Euclidean distance produced comparable or better path lengths with lower computation time, we use the Euclidean heuristic for the main planner.

3) Effect of Heuristic Weight: We also test weighted A* with different values of ϵ at $R = 0.5$. Increasing ϵ makes the search greedier, which generally reduces the number of expanded nodes but may increase path length. For example, on the flappy bird map with warm-start Euclidean A*, increasing ϵ from 1.5 to 5.0 reduces the expanded nodes from 1027 to 564, but increases the path length from 26 to 30. On the tower map, increasing ϵ from 1.5 to 5.0 reduces expanded nodes from 1626 to 295, but increases path length from 33 to 39.

This demonstrates the expected weighted A* tradeoff. Larger heuristic weights improve computation speed by making the planner more goal-directed, but the resulting path is less optimal. For part 3, we use $\epsilon = 1.5$ because it improves replanning speed while keeping the path lengths relatively close to the lower-weight solutions.

E. Part 3 Dynamic Replanning Results

For dynamic replanning, we use $R = 0.5$ and $\epsilon = 1.5$. The key modification is warm-start. Comparing cold-start and warm-start for dynamic environments, we have that for the Window environment, the Euclidean planner with $\epsilon = 1.5$ reduces total planning time from 0.2185s to 0.0909s, which is a 58.4% reduction in runtime. The number of expanded nodes also decreases from 526 to 148, a 71.9% reduction. This shows that warm-start is very effective for the Window environment.

In the Room environment, warm-start replanning reduces total planning time from 0.2273s to 0.1683s, which is a 26.0% reduction. The expanded node count decreases from 562 to 431, a 23.3% reduction. The improvement is smaller than in Window, but still significant.

The tradeoff is that warm-start does not always minimize path distance. In Window, the warm-start path distance is 176.64, while the cold-start path distance is 162.05, so the warm-start solution is about 9.0% longer. In Room, both warm-start and cold-start obtain the same total path distance of 67.43. Therefore, the planner prioritizes fast replanning and fewer expanded nodes, while accepting possible suboptimality in path quality.

The minimum achievable dynamic target period is determined by the slowest required planning time for each target hold interval. For Window at $\epsilon = 1.5$, the first goal takes 0.0109s and the slowest later goal takes 0.0517s. Therefore, the minimum period we can support is approximately

$$t_1 + 7t_2 = 0.0109 + 7(0.0517) = 0.3728 \text{ s}.$$

This is much smaller than the required Window timing period of

$$3 + 7(1) = 10 \text{ s}.$$

Thus, our planner's achievable period is 96.3% below the required period.

For Room at $\epsilon = 1.5$, the first goal takes 0.0002s and the slowest later goal is approximately 0.0408s. Therefore, the minimum period is approximately

$$t_1 + 7t_2 = 0.0002 + 7(0.0408) = 0.2858 \text{ s}.$$

This is also below the required Room timing period of

$$0.3 + 7(0.1) = 1.0 \text{ s}.$$

Thus, our planner's achievable period is 71.4% below the required period.

Overall, warm-start replanning improves time efficiency by reducing repeated graph construction and node expansion, but has higher memory usage because the graph persistance and suboptimal pathways.

F. Discussion

Overall, the planner performs well on both static and dynamic environments. The results show that collision-free paths are found for all main maps at the selected setting of $R = 0.5$. The most important computational bottleneck is the number of expanded nodes, which depends on resolution, heuristic weight, and obstacle layout.

The results also show that there is no single parameter that is best for every metric. Lower resolutions can improve path quality but greatly increase runtime. Higher heuristic weights reduce node expansions but produce longer paths. Therefore, the final parameter choice of $R = 0.5$, Euclidean heuristic, and $\epsilon = 1.5$ for dynamic replanning provides a balance between path quality and computation time. The warm-start planner improves dynamic replanning by trading memory for speed. However, for fast replanning, it is a necessary tradeoff.